

2025년 스마트해운물류 × ICT멘토링 프로젝트 결과보고서

2025. 10. 29

프로젝트명	번호	25_42
	국문	항만 유해물질 탐지 AI경비로봇
	영문	Port Hazardous Substance Detection AI Security Robot
작 품 명	항만 유해물질 탐지 AI경비로봇	
팀 명	서드임팩트	
멘 토	MBC 박진산	
팀 장	인천대학교 임베디드시스템공학과 이원종	
팀 원	인천대학교 임베디드시스템공학과 박성국 인천대학교 임베디드시스템공학과 이선우	

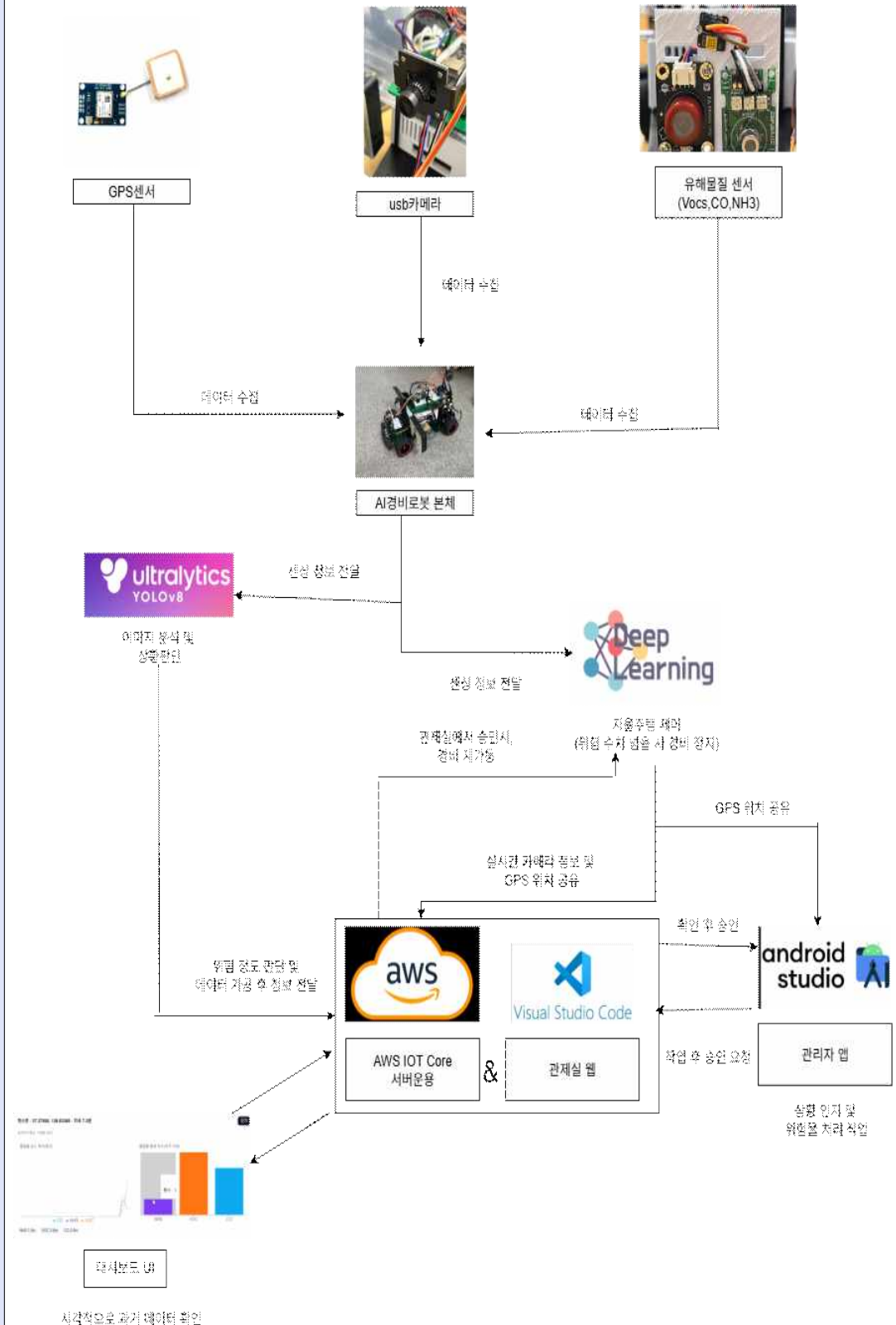
요 약 본

프로젝트 정보				
프로젝트명	항만 유해물질 탐지 AI경비로봇			
주제영역	☐	컨테이너 화물 실시간 위치 및 상태 추적	☒	항만 안전 IoT 개발
	☒	유해 화학물질 가스 누출 감지	☐	항만 원격조정 훈련 콘텐츠 제작
	☐	물류 데이터 융복합 플랫폼	☐	지속가능한 신재생에너지 활용
	☐	실시간 항만 시설물 모니터링 및 예지 정비	☐	액체 물류 스마트 모니터링 및 통제 기술
	☒	사람-항만-선박 간 통신 교통 관제	☐	해양물류항만의 적용 가능한 사업 아이템
	☐	기타(<i>기타사항 기재</i>)		
달성성과	(스마트해운물류 선발대회) ☐ 1차통과 ☐ 2차통과 ☐ 특허 ☐ 학술(논문게재)			
	☐ 창업연계 ☐ 프로그램등록 ☐ 앱등록 ☐ 기술이전			
수행기간	2025. 6. 1. ~ 2025. 10. 31.			
프로젝트소개 및 제안배경	자율주행 로봇에 VOCs, CO, NH ₃ 센서·카메라·GPS를 결합해 항만 유해물질을 실시간 감지·경보하는 시스템임. 2025년 진해 경유 유출 등 사고 사례와 인력 순찰 공백, 피로 문제로 상시 자동 감시 필요성이 큼. Jetson Orin Nano 엣지 추론과 EC2 서버(DB, MQTT/REST)를 통해 저지연 분석 및 이력화가 가능해짐. 이상 감지 시 LED, 부저, 앱 푸시, 관제 웹 대시보드로 즉시 알림하고 위치·증거 기록을 연동함. 초기 대응 시간 단축, 24시간 무중단 감시, 데이터 기반 고위험 구역 선별로 안전성과 운영 효율이 향상됨.			
주요기능	앱은 유해물질 감지 시 경고 알림과 로그 기록을 제공하며 지도에서 로봇 위치와 상황 해결 상태를 확인할 수 있음. 웹은 실시간 센서 데이터와 CCTV 화면을 시각화하고 관리자 승인·통계 리포트를 제공함. EC2 서버는 ESP32에서 수집한 센서값과 GPS 데이터를 DB에 저장하고 앱·웹으로 실시간 제공함. Jetson Orin Nano는 카메라·센서 데이터를 분석해 자율주행과 유해물질 분류를 수행함. LED·부저는 위험 등급에 따라 시각·청각 경고를 출력함.			
적용기술	Jetson Orin Nano를 활용한 딥러닝 기반 자율주행 및 유해물질 분류 기술 적용됨. ESP32와 Arduino Uno를 이용해 VOCs, CO, NH ₃ , GPS 센서 데이터를 수집하고 MQTT 통신으로 서버 전송함. AWS EC2 서버에서 Flask 기반 API와 DB를 통해 실시간 데이터 관리 및 분석 수행됨. 앱은 Flutter, 웹은 Next.js·Tailwind CSS로 구현되어 실시간 모니터링 및 알림 기능 제공함. YOLO 모델을 통한 영상 분석과 다중 센서 융합으로 유해물질 감지 정확도가 향상됨.			
기대효과 및 활용분야	항만 내 유해물질 누출 시 실시간 감지·경보로 초기 대응 시간이 단축됨. 24시간 무인 자율순찰이 가능해져 인력 의존도를 줄이고 안전성이 향상됨. 수집된 데이터로 위험지역 패턴을 분석해 예방적 감시 및 정비가 가능해짐. 환경부, 해양수산부 등 항만 관련 기관에서 안전 관리용으로 활용 가능함. 유해물질 사고가 잦은 산업단지나 도시 지역에서도 확장 적용이 가능함.			

작품 구성도

작품 정보

작품 구성도



프로젝트 결과보고서

I. 프로젝트 개요

가. 작품 소개

1) 작품명

- 딥러닝을 활용한 자율주행 자동차를 기반으로 하는 항만경비로봇임
- 특정 구역을 자율주행하며 유해물질이 유출되었는지 감지하고 빠르게 대처하기 위해 사용됨

2) 기획의도

- 위험물을 대량으로 처리하는 항만에서는 위험물에 의한 사고위험이 큼
- 항만 경비의 자동화로 현장 관리자의 근로 피로도를 줄일 필요성이 있음
- 항만 경비를 자동화하여 경비 비용을 최소화하고 앱과 웹을 통해서 유해물질 감지에 대한 빠른 대처를 목표로 함

나. 작품의 개발 배경 및 필요성

1) 개발배경

- 2025년 5월 진해 경유 유출사고로 인한 오염과 안전 취약성을 인지함
- 한국에는 아직 항만 경비에 대한 자동화가 진행되지 않아 효율성이 높지 않다는 것을 파악

2) 필요성

- 한국에는 아직 항만 경비에 대한 자동화가 진행되지 않아 24시간 경비가 힘들다는 시간적 허점이 존재함
- 사람이 경비함에따라 발생하는 인건비, 유해물질 사고 지역에 대한 구역 통제 및 시민 대피를 위해 사용되는 자원등 때문에 시간적, 경제적 비용이 큼

다. 작품의 특징 및 장점 / 국내 · 외 기술 현황 및 본 프로젝트의 차별성

1) 국내 · 외 기술 현황

- 부산 스마트 항만 프로젝트 : IoT 와 AI 기술활용, 항만 운영 효율화 및 안전성 강화
- 인천항 항만물류기술개발 사업 : AI와 빅데이터로 스마트 물류 시스템 구축, 화물 처리 속도 개선
- 광양항 스마트 항만 구축 프로젝트 : 자동화 테스트베드 구축 및 스마트 기술 실증
- 싱가포르 PSA 항만 자동화 프로젝트 : AI와 자율주행 기술로 컨테이너 관리, 세계

최대 자동화 항만 운영

- 미국 SMP Robotics Rover S5 Hazmat : 위험물질 감지를 위한 자율주행 로봇, 화학 공장 및 항만에서 사용

2) 작품의 특징 및 장점(차별성)

- 주로 실시간 감지와 기본적인 모니터링을 통한 데이터 수집을 목표로 하는 기존 시스템과 달리 다중센서를 통한 여러 유해물질 감지와 더불어 앱을 통해서 항만 종사자가 빠른 대처를 할 수 있을 뿐만 아니라 일반 시민들 또한 위험지역을 확인하고 대피하도록 할수 있음

II. 프로젝트 수행결과

가. 프로젝트 개발환경

구분		상세내용
S/W 개발환경	OS	Linux
	개발환경(IDE)	Arduino IDE, Android Studio IDE, Node.js
	개발도구	Visual studio code, Android Studio, VS Code, Flutter, FCM, Next.js, Tailwind CSS, Flask,
	개발언어	Python, cpp, Dart, TypeScript, JavaScript
	기타사항	
H/W 구성장비	디바이스	Arduino Uno, jetson Nano Orin super, YB-ERF01-V3.0 with STM32, ESP32-DEVKITC-32UE
	센서	GPS센서(SZH-NT07), VOC센서(GSBT11-P110), NH3센서(MQ-7, SEN0132), CO센서(MQ-7, SEN0132)
	통신	Serial, network
	언어	C, python
	기타사항	
프로젝트 관리환경	형상관리	브랜치 전략 - main: 배포 가능한 안정 버전 보관함. 항상 신뢰 가능해야 함. - dev: 통합 개발용 브랜치임. 기능을 모아 테스트·안정화 후 main으로 승격함. - feature/* : 새 기능 단위 개발 브랜치임(예: feature/add-night-led). 완료 시 dev로 PR 올림. - hotfix/* : 운영 중 긴급 버그 수정용임. main에 우선 반영 후 dev에도 동일 병합함. PR(병합) 절차 - feature/hotfix에서 작업 완료 → PR 생성함. - 코드리뷰 1인 이상 승인 + CI 통과되면 병합함. - 릴리스 시 dev→main 병합하고 태그(예: v0.3.0) 부여함. 커밋 규칙(Conventional Commits) - 형식: <type>(<scope>): <짧은 설명> - 주요 type: feat(기능), fix(버그), docs(문서), refactor(리팩터), perf(성능), test(테스트), chore(설정)
	의사소통관리	카카오톡 채팅, Notion, 주간 미팅(google meet) 회의 마다 자신의 활동을 발표
	기타사항	

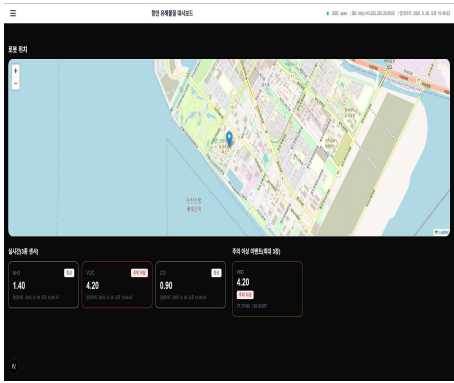
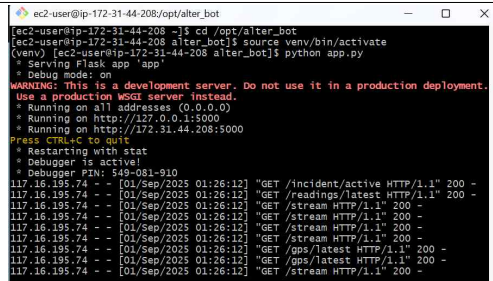
나. 환경장비(기자재/재료) 활용

번호	품명	작품에서의 주요기능
1	아두이노 우노	- 각종 아날로그 센서의 정보를 수집 및 (ESP32를 통해) 데이터베이스에 저장
2	젯슨 오린 나노	- 자율주행, 위험물 감지시 영상 처리
3	GPS 센서	- 현재 위치를 확인
4	VOCs 센서	- VOCs 농도를 확인
5	NH3 센서	- 암모니아 농도를 확인
6	CO 센서	- 일산화 탄소 농도를 확인

다. 주요 기능 목록

구분	기능	설명	현재진척도(%)
S/W	앱	유해물질 감지로 인한 위험 상황 발생시 (관리자 활용) - 상황 발생 위치 공유 - 상황 처리 메뉴얼을 제공	100
	웹	유해물질 감지로 인한 위험 상황 발생시 (관제실 활용) - 실시간 유해물질 감지 농도 확인 - 관리자의 현장 처리 상황을 확인 - DB에 있는 데이터를 가공하여 통계 자료 제공 - 실시간 CCTV 화면 제공 완료 가능 시점 (9월/15일)	90
	Ec2서버	구매 S/W 위험 상황이 있었던 기존 데이터들을 모아서 DB에 저장	100
	유해물질 감지 (YOLO)	이미지 분석을 통한 유해물질 종류 감지 완료가능시점 (9월/15일)	20
H/W	VOcs, CO, NH3, GPS센서	구매 H/W 유해 물질 감지 및 현재 위치 확인	100
	ESP32-DEVKITC-32UE	구매 H/W 아두이노가 처리한 아날로그 신호(각종 센서 신호) 를 Ec2서버로 전송	100
	Jetson Orin Nano Super	구매 H/W - 딥러닝을 통한 자율주행 - YOLO 모델을 이용한 추론으로 유해물질 종류 확인	100
	Arduino Uno	구매 H/W 각종 센서들에서 나오는 아날로그 신호 처리	100
	YB-ERF01-V3.0 with STM32	구매 H/W - 서보 모터 조작 (조향각 설정) - 모터 속도 설정 - 부저 제어 - 전원 관리	100
	LED&camera&부저	구매 H/W - camera : 자율주행 구현 및 실시간 CCTV에서 사용	100

1) S/W 개발 기능 상세내용

기능	설명	작품실물사진
앱 (모바일)	유해물질 감지 시 경고 알림과 로그 기록을 제공하며, 지도에서 로봇 위치 확인과 해결 상태 전송 기능을 지원함. 또한 상황발생 지점에 대한 방향 지원과 상황대비 숙지 메뉴얼이 있음	
웹	실시간 센서 데이터를 시각화 하고 주의 이상에 대한 이벤트 승인과 승인을 하기 전 이중승인을 빌미로 앱에서 보내온 증거 기록을 관리함. 이벤트들의 통계 분석 기능 또한 포함되어 있음	
EC2서버	AWS EC2 인스턴스를 통해서 서버를 열고 서버에서 Esp32네트워크 모듈에게 센서값을 받고 데이터 베이스에 저장한다 필요에 따라 앱과 웹등에 데이터베이스에 저장된 값을 보내준다.	

- S/W 주요 목표대비 현재 개발결과 상세 설명

1. 앱 (모바일)

목표:

- 유해물질 감지 시 경고 알림 발송함. 지도에서 로봇 위치 · 사건 지점 확인.
- 사건 상태 전송(위험 상황 알림→이중 승인 대기→해제)과 대응 매뉴얼 지원
- 나침반으로 방향 안내 제공.
- 현장 증거(사진/메모/동영상) 업로드 지원.

현재: 구현 완료.

2. 웹

목표:

- 실시간 센서 시각화(VOCs/CO/NH₃ , 지도/타임라인) 제공.
- 사건 승인 처리(앱 증거 확인 후 승인) 운영함.

- 기간 · 구역 · 센서별 통계/리포트 제공함.
- 실시간 CCTV 패널 제공함.

현재: 대시보드(그래프 · 지도) 동작됨. 증거 기반 승인/반려 기능 동작됨. 통계/리포트 생성됨(CSV). CCTV 패널 UI 슬롯 마련됨.

구현 중: 실시간 CCTV 정보를 jetson Orin Nano로부터 받아오는 것 구현 중.

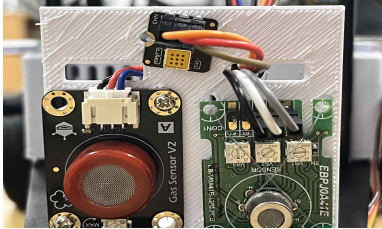
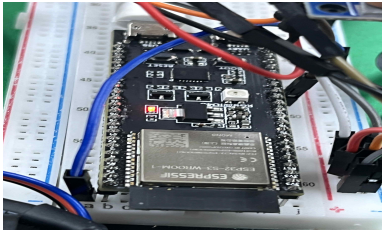
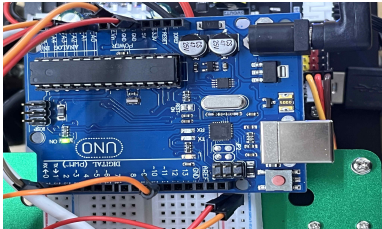
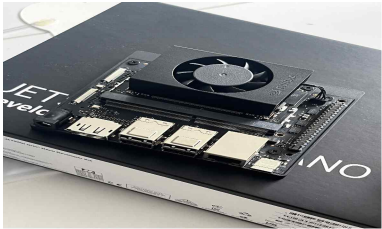
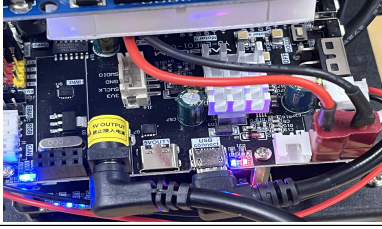
3. EC2 서버 (수집 · 저장 · API)

목표:

- ESP32 네트워크 모듈에서 수집한 센서값 · GPS를 안정 수신하여 DB 저장함.
- 앱/웹에 최근값 · 이벤트 · 통계 API 제공함. 보안그룹/IAM, 백업/모니터링, MQTT(TLS) 브로커/브리지 운용함.

현재: 구현 완료.

2) H/W 개발 기능 상세내용

기능/부품	설명	작품실물사진
VOCs,CO,NH3,GPS 센서	Arduino Uno에의해 명령받아 각 VOCs,CO,NH3 기체의 농도를 측정. GPS 센서의 경우 해당 위치를 측정.	
ESP32-DEVKITC-32 UE	측정한 VOCs,CO,NH3,GPS 값을 Arduino Uno에서 전달받고 전달받은 값을 서버로 보냄	
Arduino Uno	- 센서들을 통해 해당값을 측정 - 측정한 값을 Serial을 통해 esp32네트워크 모듈로 보냄	
Jetson Orin Nano Super	- 센서와 카메라 데이터를 받고 그 정보를 기반으로 딥러닝 학습을 통해 자율주행을 구현 - 서버와 통신을 통해 실시간 CCTV를 공유 - YOLO기반의 이미지 판단을 통한 위험 물질 분류	
LED&부저&camera	- LED와 부저는 정해진 시간 간격등을 통해 위험하다는 신호를 알림 - camera의 경우는 자율주행과 실시간 모니터링을 위해 사용	
YB-ERF01-V3.0 with STM32	- LED의 밝기, 깜빡이는 간격, 색 등을 제어하여 위험을 알림 - 부저의 소리의 세기 등을 제어해 위험을 알림	

- H/W 주요 목표대비 현재 개발결과 상세 설명

1. VOCs · CO · NH₃ · GPS 센서 (Arduino Uno + 센서 보드)

목표: 각 센서의 농도(ppm/ppb)와 위치(GPS) 안정 취득

현재: Arduino에서 주기 샘플링 및 단위 변환 수행됨. GPS 좌표 취득됨. 시리얼로

ESP32에 전달됨.

2. ESP32-DEVKITC-32UE (네트워크 모듈)

목표: Arduino로부터 수신한 값(VOCs/CO/NH₃ /GPS)을 MQTT(TLS)로 서버에 안정 전송,

현재: 시리얼 수신 · 파싱 및 값 전달 루틴 구현됨(초기 전송 경로 동작됨).

3. Arduino Uno (센싱 · 전처리 컨트롤러)

목표: 센서 통합, 고정 샘플링 레이트, 시리얼 프로토콜 표준화

현재: 센서 값 취득 및 시리얼 송신 동작됨.

3. Jetson Orin Nano Super (엣지 컴퓨팅/자율주행/추론)

목표: 카메라 · 센서 수집, TensorRT 기반 저지연 추론, 자율주행(조향 필터 · 슬루), 관제 연동, 실시간 CCTV, 유해물질 종류 분류 구현

현재: 도로 추종 TRT 추론 + EMA + 슬루 제한으로 저속 크루즈 동작됨.

유해 물질 감지시 정지 후 서버에서 승인 완료 시 다시 경비 시작

구현 중: 실시간 CCTV 스트리밍(WebRTC/RTSP→HLS) 구현 중.

YOLO를 통한 유해물질 종류 분류 구현 중.(시각/가스 멀티모달 결합 필요).

GStreamer 파이프라인 확정 → WebRTC 송출(지연 < 500 ms 목표) 구현 중.

4. LED & 부저 & 카메라 (현장 경보/야간 운용)

목표: 위험에 따른 광 · 음 패턴(색/주기/음압) 제공, 야간 시야 확보(LED 라이트바/작업등), 자율주행 · 모니터링용 영상 확보함.

현재: LED · 부저 제어 루틴 동작됨, 야간 LED 운용 가능함. 카메라는 자율주행 입력에 사용됨.

구현 중: 야간 운용용 LED 자동화

5. YB-ERF01-V3.0 with STM32 (확장 보드)

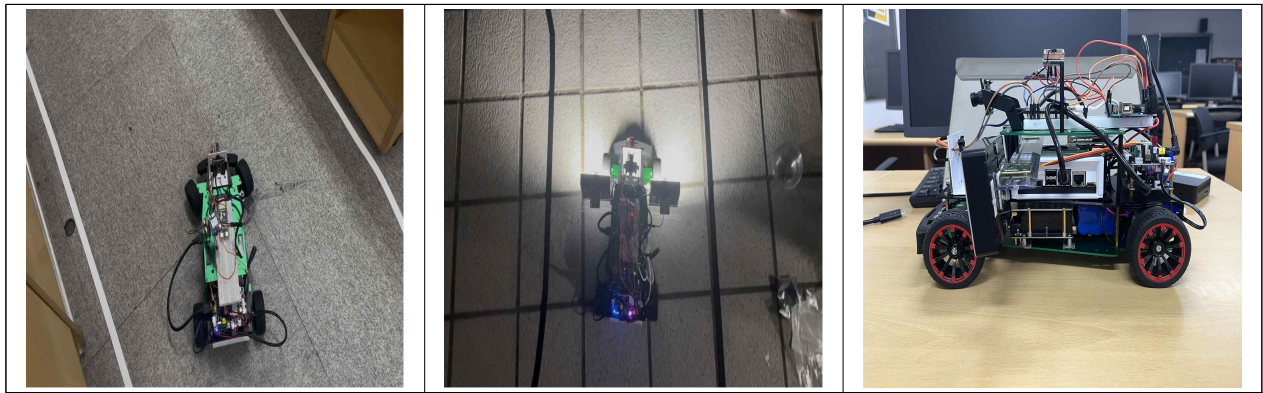
목표: 경보용 LED 밝기/패턴, 부저 음압/톤을 명령 세트로 수신 · 실행, 모터 제어

현재: 기본 명령에 따른 밝기/깜빡임/색, 부저 세기 제어 동작됨. 서보 모터를 통한 조향각 제어, 모터를 통한 속도 제어

라. 프로그램 작동 동영상

<https://youtu.be/er13wU-xJRw?si=iQRGlV7aXJ4JHtw>

마. 결과물 상세 이미지



바. 달성성과

☐ 논문게재 및 포스터발표	게재(발표)자명	논문(포스터)명	게재(발표)처	게재(발표)일자
				2025. 00. 00.
☐ 앱(APP) 등록	등록자명	앱(APP)명	등록처	등록일자
				2025. 00. 00.
☐ 프로그램 등록	등록자명	프로그램명	등록처	등록일자
				2025. 00. 00.
☐ 특허출원	출원자명	특허명	출원번호	출원일자
				2025. 00. 00.
☐ 공모전	구분(교내/대외)	공모전명	수상여부(출품/수상)	상격
☐ 실용화	#실용화한 내용에 대한 구체적 작품설명			
☒ 기타	취업 시 포트폴리오로 활용			

Ⅲ. 프로젝트 수행방법

가. 업무분장

번호	성명	역할	담당업무
1	박진산	멘 토	매월 회의 개최 및 수정사항 제시, 방향성 제공
2	이원종	팀 장	프로젝트 관리 및 자율 주행 구현
3	방성국	팀 원2	앱과 웹 구현
4	이선우	팀 원3	데이터베이스 및 다양한 센서 구축

나. 프로젝트 수행일정

프로젝트 기간			2025. 06. 01. ~ 2025. 10. 31.				
구분	추진내용	구분	프로젝트 기간				
			6월	7월	8월	9월	10월
계획	해운 물류 관련 위험물 탐색 및 해결방안 모색	계획					
		진행					
분석	경비로봇 전체적 구상 및 및 관련 자료 조사 및 스터디	계획					
		진행					
설계	H/W 설계 및 관련 역할 분담	계획					
		진행					
	S/W 설계 및 관련 역할 분담	계획					
		진행					
개발	자율 주행 제어 관련 개발	계획					
		진행					
	센서 및 서버 개발	계획					
		진행					
	데이터 처리, 모니터링 관련 개발	계획					
		진행					
테스트	작동과정 테스트 및 피드백	계획					
		진행					
종료	성과 정리, 프로젝트 문서화	계획					
		진행					

다. 프로젝트 수행(협업) 방안

- 팀 프로젝트를 수행하면서 팀원들과 각자 만날 수 있는 시간을 확인하고 만날 수 있는시간에는 특정장소(학교)에 모여서 각자의 일을 수행함
- 전부 만날 수 없는 날에는 각자 집에서 일을 수행하다가 일주일에 한번씩 google meet를 통해서 진행 상황을 공유함
- 한 달에 한 번씩 멘토님과 질의응답 및 조언을 얻는 시간을 가짐

라. 문제점 및 해결방안

1) 프로젝트 관리 측면

- 기숙사의 정책상 기숙사를 비워야하는 기간이 존재하게 되어 혼자 먼 지역에서 살고있는 한 팀원 때문에 지속적으로 만나서 프로젝트를 수행함에 있어 어려움을 겪음.
- 팀원간의 역할을 정해서 각자 맡은 바를 수행하다가 모두 만날 수 있는 하루를 정하고 모두 만나는 날에 밤샘 작업을 수행함으로써 문제를 해결함.

2) 작품 개발 측면

- 장비 사용 미숙으로 인한 하드웨어 고장이라는 문제가 발생하였고 이를 알아채는데 늦어 많은 시간이 낭비되는 문제가 발생함
- 같은 기능을 하는 새로운 장비를 구매 후 고장난 장비와 기능적으로 비교하여 차이점을 확인하고 이를 기준으로 고장난 장비인지 아닌지를 판단하는 기준을 세움.
- 또한 하드웨어를 고장난 위험한 행동을 반복하지 않도록 기존 방식이 아닌 안전한 새로운 방식을 찾아서 개발에 진행함.

IV. 기대효과 및 활용분야

가. 작품의 활용(적용)분야

- 환경부, 해양수산부 등 항만과 관련하여 종사하는 관리자 분들, 관제실 근로자 분들이 사용하는 것을 목표로 함
- 항만 뿐만 아니라 유해물질 유출 사고가 자주 일어나는 지역에서도 활용되는 것을 목표로 함
- 유해물질 사고지역을 알려주는 앱의 일부 기능들은 항만 종사자 뿐만 아니라 유해물질 사고가 자주 일어나는 지역 시민들을 위해서 사용되는 것을 목표로 함

나. 작품의 활용에 의한 기대효과(사용자)

- 유해물질 누출 및 이상상황 발생 시, 실시간 자동 경고 및 관리자 보고를 통해 초기 대응 시간 단축
- 기존 인력 감시 방식의 한계를 보완하여, 24시간 무중단 감시체계 구축
- 반복되는 순찰 데이터를 기반으로 위험도 분석 및 고위험 지역 선별 감시로 감시 효율성 향상
- 앱을 통해서 시민들을 대피시킬 수 있으므로 지역 통제 및 시민 대피 비용을 절감

다. 작품의 기대가치

- 다중 센서 융합 감지(NH3/VOC/CO)로 유해물질 유출을 조기 탐지하고 교차 검증하여 오탐·미탐을 최소화함.
- 앱·웹 실시간 알람과 지도 기반 대피 경로 안내로 골든타임 내 대응을 지원해, 인명·재산 피해를 최소화함.
- 24시간 자율주행 순찰 및 관제 연동으로 경비 효율을 높이고, 인력 중심. 모니터링 대비 운영비를 절감함.
- 누적 데이터 분석으로 시간대·구역별 핫스팟을 도출해 재발 방지 정책과 시설 개선에 근거를 제공함.
- 모듈형 아키텍처로 센서·로봇 플랫폼 확장이 용이해 항만을 넘어 산업단지·물류창고 등으로 손쉽게 확장 가능함.

V. 참고자료

가. 참고 및 인용자료

- [yahboom 사이트](#)
- [nvidia 질문 사이트](#)

별첨

스마트해운물류 × ICT멘토링 프로젝트 산출물

스마트해운물류×ICT멘토링 프로젝트 산출물

1. 센서, 서버 관련 코드

.....

18

2. 앱, 웹 관련 코드

.....

21

3. 자율주행 관련 코드

.....

30

별첨

센서, 서버 관련 코드

● wifi_code.ino

```
#include <WiFi.h>
#include <HTTPClient.h>

const char* ssid = "Use wifi name";
const char* password = "use wifi-passwords";
const char* HAZ_URL = "http://server_ip:port/data/hazard";
const char* DATA_URL = "http://server_ip:port/data";
#define RXD1 10
#define TXD1 11

void setup() {
  Serial.begin(115200);
  Serial1.begin(9600, SERIAL_8N1, RXD1, TXD1);
  Serial1.setTimeout(2000);
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) { delay(500); }
  Serial.println("WiFi 연결됨");
}

void loop() {
  if (!Serial1.available()) return;
  String data = Serial1.readStringUntil('\n');
  data.trim();
  Serial.println(data);
  int vocsIdx = data.indexOf("VOCs=");
  int nh3Idx = data.indexOf("NH3=");
  int coIdx = data.indexOf("CO=");
  int gpsIdx = data.indexOf("GPS=");
  if (vocsIdx== -1 || nh3Idx== -1 || coIdx== -1 || gpsIdx== -1) return;
  String vocs = data.substring(vocsIdx + 5, nh3Idx - 1);
  String nh3 = data.substring(nh3Idx + 4, coIdx - 1);
  String co = data.substring(coIdx + 3, gpsIdx - 1);
  String gps = data.substring(gpsIdx + 4);
  gps.trim();
  // GPS "lat,lng" 추출 (마지막 글자 잘리지 않게!)
  int comma = gps.indexOf(',');
  if (comma & 0) return;
  String lat = gps.substring(0, comma); // &- lngidx-1 아니고 comma까지 그대로
  String lng = gps.substring(comma + 1);
  lat.trim(); lng.trim();
  // 1) GPS 먼저 전송 (/data, type=GPS, value="lat,lng")
  postJson(DATA_URL, String("{\"type\":\"GPS\",\"value\":\"" + lat + "," + lng + "\"}"));
  // 2) 센서값은 /data/hazard 로 전송 (lat/lng 반드시 포함)
```

```
    sendHaz("VOC", vocs, lat, lng);
    delay(300);
    sendHaz("NH3", nh3, lat, lng);
    delay(300);
    sendHaz("CO", co, lat, lng);
    delay(300);
}

void sendHaz(const char* typ, String val, String lat, String lng) {
    val.trim();
    String body = String("{\"substance\":\"" + typ +
        "\",\"concentration\":\"" + val +
        "\",\"lat\":\"" + lat +
        "\",\"lng\":\"" + lng + "\"}");
    postJson(HAZ_URL, body);
}

void postJson(const char* url, const String& json) {
    HTTPClient http;
    http.begin(url);
    http.addHeader("Content-Type", "application/json");
    int code = http.POST(json);
    Serial.printf("[POST %s] %d\n", url, code);
    http.end();
}
```

● sensor_code.ino

```
#include <SoftwareSerial.h>
#include <TinyGPS++>
TinyGPSPlus loc;
SoftwareSerial gps(11,12);

void setup() {
    // put your setup code here, to run once:
    Serial.begin(9600);
    pinMode(A0,INPUT);
    pinMode(A1,INPUT);
    pinMode(A2,INPUT);
    gps.begin(9600);
}

void loop() {
    // put your main code here, to run repeatedly:
    while(gps.available()){
        loc.encode(gps.read());
    }
    analogRead(A0);
}
```

```
int vpin = analogRead(A0);  
analogRead(A1);  
int Npin = analogRead(A1);  
analogRead(A2);  
int cpin = analogRead(A2);  
float vppm = vpin/1023.0 * 5.0;  
float nppm = Npin/1023.0 * 5.0;  
float cppm = cpin/1023.0 * 5.0;  
Serial.print("VOCs=");Serial.print(vppm);  
Serial.print(",NH3=");Serial.print(nppm);  
Serial.print(",CO=");Serial.print(cppm);  
Serial.print(",GPS=");Serial.print(loc.location.lat(),6);  
Serial.print(",");Serial.println(loc.location.lng(),6);  
delay(100);  
}
```

별첨 앱, 웹 관련 코드

● (웹) 실시간 모니터링, 대시보드, 안정적 폴링, 백오프, 상태관리 관련 코드 (pages/index.tsx)

```
// pages/index.tsx
import React, { useEffect, useState } from "react";
import dynamic from "next/dynamic";
import { useRouter } from "next/router";
import useIncident, { Incident } from "../frontend/hooks/useIncident";
import ActiveIncidentGrid from "../frontend/components/ActiveIncidentGrid";
import LiveReadings from "../frontend/components/LiveReadings";
import EventStatus from "../frontend/components/EventStatus";
import StatsGeoBoard from "../frontend/components/StatsGeoBoard";
const RobotMap = dynamic(() => import("../frontend/components/RobotMap"), { ssr: false });
const BE = (process.env.NEXT_PUBLIC_BACKEND_URL as string) || "http://localhost:5000";
const TOKEN = process.env.NEXT_PUBLIC_ADMIN_TOKEN as string;
type ReadingMsg = {
  id: number;
  type: "NH3" | "VOC" | "CO" | "GPS";
  value?: number | null;
  lat?: number | null;
  lng?: number | null;
  vehicle_id?: string;
  timestamp: string;
};
export default function Home() {
  const router = useRouter();
  const [menu, setMenu] = useState<"live" | "stats" | "admin">("live");
  const [open, setOpen] = useState(false);
  const { incident, setIncident, state } = useIncident(BE);
  // 실시간 센서 최신값 저장 (주의 이하도 모두 수신)
  const [latestReadings, setLatestReadings] = useState<Record<string, ReadingMsg | null>>({
    NH3: null, VOC: null, CO: null,
  });
  // (선택) 최신 GPS
  const [latestGps, setLatestGps] = useState<ReadingMsg | null>(null);
  useEffect(() => {
    const es = new EventSource(`${BE}/stream`, { withCredentials: false });
    // GPS 최초/갱신
    es.addEventListener("gps", (ev) => {
      const msg = JSON.parse((ev as MessageEvent).data) as ReadingMsg;
      setLatestGps(msg);
    });
  });
```

```
es.onerror = () =& {
  // 연결이 끊겨도 브라우저가 자동 재연결 시도
  // 필요 시 로그인만
  // console.warn("SSE disconnected");
};
return () =& es.close();
}, []);
const approve = async (id: number) =& {
  const r = await fetch(`${BE}/events/${id}/approve?token=${TOKEN}`, {
    method: "POST",
    headers: { "X-Admin-Token": TOKEN, "X-User": "web" },
  });
  if (!r.ok) {
    const t = await r.text();
    alert(`승인 실패 ${r.status}: ${t}`);
    return;
  }
  setIncident(null);
};
return (
  &div className="min-h-screen"&
    { /* 헤더 */ }
    &header className="fixed top-0 inset-x-0 h-12 px-4 bg-white text-gray-900 border-b
z-[1100] flex items-center justify-between"&
      &button onClick={() =& setOpen(true)} className="p-2 rounded hover:bg-gray-100"
aria-label="open menu"&
        &span className="block w-5 h-0.5 bg-black mb-1" /&
        &span className="block w-5 h-0.5 bg-black mb-1" /&
        &span className="block w-5 h-0.5 bg-black" /&
      &/button&
      &div className="font-semibold"&항만 유해물질 대시보드&/div&
      &EventStatus state={state} /&
    &/header&
    { /* 드로어 */ }
    {open && (
      &div className="fixed inset-0 z-[2000]"&
        &div className="absolute inset-0 bg-black/40" onClick={() =& setOpen(false)} /&
        &aside className="absolute left-0 top-0 bottom-0 w-64 bg-white text-gray-900 p-4
space-y-2 shadow-2xl"&
          &h2 className="text-sm font-bold mb-2"&메뉴&/h2&
          &NavBtn active={menu === "live"} onClick={() =& { setMenu("live"); setOpen(false); }}&
라이브&/NavBtn&
          &NavBtn active={menu === "stats"} onClick={() =& { setMenu("stats"); setOpen(false);
}}&통계&/NavBtn&
```

```
&NavBtn active={menu === "admin"} onClick={() => { setMenu("admin"); setOpen(false);
}}&초기화&/NavBtn&
    &NavBtn active={false} onClick={() => { router.push("/logs"); setOpen(false); }}&해결 로
그&/NavBtn&
    &/aside&
  &/div&
)}
{/* 본문 */}
&main className="pt-12 p-4 space-y-6"&
  {menu === "live" && (
    &&
    &section&
      &h2 className="text-lg font-semibold mb-2"&로봇 위치&/h2&
      { /* RobotMap이 내부에서 SSE를 보지 않는다면 latestGps를 prop으로 넘겨도 됨 */}
      &RobotMap /* gps={latestGps} */ /&
    &/section&
    &section className="grid grid-cols-1 lg:grid-cols-2 gap-6"&
      &div className="space-y-3"&
        &h3 className="text-base font-semibold"&실시간(3종 센서)&/h3&
        &LiveReadings /&
      &/div&
      &div className="space-y-3"&
        &h3 className="text-base font-semibold"&주의 이상 이벤트(최대 3장)&/h3&
        &ActiveIncidentGrid incident={incident as Incident | null} onApprove={approve} /&
      &/div&
    &/section&
  &/&
)}
{menu === "stats" && (
  &div className="space-y-4"&
    &h2 className="text-lg font-semibold"&통계&/h2&
    &StatsGeoBoard /&
  &/div&
)}
{menu === "admin" && (
  &div className="space-y-4"&
    &h2 className="text-lg font-semibold"&초기화&/h2&
    &div className="flex gap-2 flex-wrap"&
      &button
        onClick={async () => { await fetch(`${BE}/admin/clear_recent?hours=24`, { method:
"POST" }); location.reload(); }}
        className="px-4 py-2 rounded-lg bg-amber-600 text-white text-sm"
      &
      &최근 24시간 서버 통계 초기화
```

```
&/button&
&button
  onClick={async () => {
    if (!confirm("모든 데이터 삭제")) return;
    await fetch(`${BE}/admin/clear_all`, { method: "POST" });
    location.reload();
  }}
  className="px-4 py-2 rounded-lg bg-red-700 text-white text-sm"
&
  서버 전체 초기화
&/button&
&button
  onClick={() => { try { localStorage.clear(); sessionStorage.clear(); } catch {} ;
location.reload(); }}
  className="px-4 py-2 rounded-lg bg-gray-700 text-white text-sm"
&
  브라우저 캐시 초기화
&/button&
&/div&
&/div&
  )}
&/main&
&/div&
);
}
function NavBtn({
  active, onClick, children,
}: { active: boolean; onClick: () => void; children: React.ReactNode; }) {
  return (
    &button
      onClick={onClick}
      className={`w-full text-left px-3 py-2 rounded-lg text-sm ${
        active ? "bg-gray-200 text-gray-900" : "text-gray-900 hover:bg-gray-100"
      }`}
    &
      {children}
    &/button&
  );
}
```


● (웹) 경고 카드 UI, 직관적 시각화, 상태 표현력 관련 코드 (components/ActiveIncidentCard.tsx)

```
// frontend/components/ActiveIncidentCard.tsx
"use client";
import React from "react";
import type { Incident } from "../hooks/useIncident";
const CAUTION = 2.0;
const DANGER = 4.0;
type Level = "danger" | "warn" | "normal";
const sev = (v: number): Level => (v >= DANGER ? "danger" : v >= CAUTION ? "warn" : "normal");
const sevBadge = (l: Level) =>
  l === "danger" ? "bg-red-100 text-red-800"
  : l === "warn" ? "bg-amber-100 text-amber-800"
  : "bg-gray-100 text-gray-700";
const sevLabel = (l: Level) => (l === "danger" ? "위험" : l === "warn" ? "주의" : "정상");
// 칩을 값 기준으로 색상 지정
const chipClassByValue = (v: number) =>
  v >= DANGER
    ? "border-red-500 text-red-700"
    : v >= CAUTION
    ? "border-amber-500 text-amber-700"
    : "border-sky-500 text-sky-700";
type Props = {
  incident: Incident | null;
  onApprove: (id: number) => void;
};
export default function ActiveIncidentCard({ incident, onApprove }: Props) {
  if (!incident)
    return <div className="p-4 rounded-xl border text-sm text-gray-500">활성 인시던트 없음</div>;
  const subs = incident.substances || [];
  const maxAny = subs.length ? Math.max(...subs.map((s) => s.max)) : 0;
  const level: Level = sev(maxAny);
  return (
    <div className="p-4 rounded-xl border flex items-center justify-between">
      <div className="min-w-0">
        <div className="flex items-center gap-2 mb-1">
          <div className="font-semibold">이벤트 · {incident.status}</div>
          <span className={`text-xs px-2 py-0.5 rounded ${sevBadge(level)}`}>
            {sevLabel(level)}{maxAny ? ` · ${maxAny.toFixed(2)}` : ""}
          </span>
        </div>
      </div>
    </div>
  );
}
```

```
&div className="flex flex-wrap gap-2 mb-1"&
  {subs.map((x) => (
    &span
      key={x.substance}
      className={`px-2 py-1 rounded-md text-xs font-semibold border
${chipClassByValue(x.max)} `}
      title={` ${x.substance} max`}
    &
      {x.substance} {x.max.toFixed(2)}
    &/span&
  ))}
&/div&
&div className="text-xs text-gray-500"&
  {incident.lat.toFixed(5)}, {incident.lng.toFixed(5)}
&/div&
&/div&
&div className="flex gap-2 shrink-0"&
  {incident.status === "pending" && (
    &button
      onClick={() => onApprove(incident.id)}
      className="px-3 py-1 rounded-lg bg-blue-600 text-white text-sm"
    &
      승인
    &/button&
  )}
&/div&
&/div&
);
}
```

● (앱) 모바일 메인 UX/데이터 구조관련 코드(main.dart)

```
import 'package:flutter/material.dart';
import 'screens/incident_screen.dart';
import 'services/push_service.dart';
import 'package:firebase_core/firebase_core.dart';
import 'firebase_options.dart';

void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Firebase.initializeApp(
    options: DefaultFirebaseOptions.currentPlatform, // ★ 중요
  );
  await PushService.init();
  runApp(const App());
}

class App extends StatelessWidget {
  const App({super.key});
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: '현장 처리',
      theme: ThemeData(useMaterial3: true, colorSchemeSeed: const Color(0xFF111827)),
      home: const IncidentScreen(),
    );
  }
}
```

● (앱) Flask 통신 게이트웨이 네트워크 안정성, 구조적 완성도 관련 코드 (api.dart)

```
import 'dart:convert';
import 'package:http/http.dart' as http;
import '../config.dart';
import '../models/hazard_event.dart';
import '../models/sensor_reading.dart';
import '../models/comment.dart';
class Api {
  static Uri _u(String p, [Map<String, String>? q]) =&
    Uri.parse('${kBackendBaseUrl$p').replace(queryParameters: q);
  static Future<List<HazardEvent>> latestMulti() async {
    final r = await http.get(_u('/latest_multi'));
    if (r.statusCode != 200) return [];
    final List arr = jsonDecode(r.body);
    return arr.map((e) =& HazardEvent.fromJson(e)).toList();
  }
  static Future<HazardEvent?> getEvent(int id) async {
    final r = await http.get(_u('/event/$id'));
    if (r.statusCode != 200) return null;
    return HazardEvent.fromJson(jsonDecode(r.body));
  }
  static Future<bool> approve(int id) async {
    final r = await http.post(_u('/approve/$id'));
    return r.statusCode == 200;
  }
  static Future<SensorReading?> latestSensor(String type) async {
    final r = await http.get(_u('/want/$type'));
    if (r.statusCode != 200) return null;
    return SensorReading.fromJson(jsonDecode(r.body));
  }
  static Future<List<CommentRow>> getComments({
    int? eventId,
    int? afterId,
  }) async {
    final q = <String, String>{};
    if (eventId != null) q['event_id'] = '$eventId';
    if (afterId != null) q['after_id'] = '$afterId';
    final r = await http.get(_u('/comments', q));
    if (r.statusCode != 200) return [];
    final List arr = jsonDecode(r.body);
    return arr.map((e) =& CommentRow.fromJson(e)).toList();
  }
}
```

```
static Future<bool> postComment({
  int? eventId,
  required String author,
  required String message,
}) async {
  final r = await http.post(
    _u('/comments'),
    headers: {'Content-Type': 'application/json'},
    body: jsonEncode(
      {'event_id': eventId, 'author': author, 'message': message},
    ),
  );
  return r.statusCode == 200;
}

static Future<void> registerPushToken(String token) async {
  await http.post(
    _u('/register_push_token'),
    headers: {'Content-Type': 'application/json'},
    body: jsonEncode({'token': token, 'platform': 'android'}),
  );
}
}
```

별첨

자율 주행 관련 코드

● auto_drive_with_buzzer_rgbbar.py

```
#!/usr/bin/env python3
# coding: utf-8
"""
START = 저속(최대속도 10%) 자율주행 ON/OFF
L2/R2 = 브레이크(정지)
- TRT 모델 출력: 스칼라  $x \in [-1, 1]$  로 가정 (tanh형)
  각도(deg) =  $x * \text{ANGLE\_LIMIT}$ 
- utils.preprocess가 있으면 그대로 사용하여 학습 파이프라인과 동기화
- '확확' 방지: 각도 EMA + 슬루(rate) 제한
- START로 켜 줄 때 0°에서 잠깐 유지 후 부드럽게 램프업
[추가]
- 브레이크 시 경고: 부저 0.3초/1초 주기, RGB 바 (빨→주→노) 1초씩 순환
- START로 자율주행 재개 시 부저/조명 OFF
"""
# ===== 설정 =====
DEFAULTS = {
    # 경로/장치
    "TRT_PATH": "road_following_model_trt.pth",
    "ALT_TRT_PATH": "/mnt/data/road_following_model_trt.pth", # 업로드 경로 대안
    "CAM_INDEX": 0, # /dev/video0
    "SERIAL_PORT": "/dev/ttyCH341USB0", # 포트를 명시하면 더 안정적
    "JS_INDEX": 0, # /dev/input/js0
    # 런타임
    "FPS": 30,
    "SEND_RATE_HZ": 20.0,
    "DEBUG": True,
    # 조향 제한/필터
    "ANGLE_LIMIT": 44, # ±44°
    "ANGLE_LPF_ALPHA": 0.35, # 각도 EMA(0이면 끄; 0.08~0.15 권장)
    "ANGLE_SLEW_DEG_PER_S": 300.0, # 각도 최대 변화속도(°/s)
    "START_NEUTRAL_SEC": 0.6, # 켜 직후 0° 유지 시간
    # 전처리/정밀도
    "TRT_FP16": True,
    "USE_UTILS_PREPROCESS": True, # utils.preprocess 있으면 최우선 사용
    "FALLBACK_BGR2RGB": True, # fallback 전처리에서만 적용
    "FALLBACK_NORM": "none", # 'none' | 'imagenet'
    # 방향 보정
    "STEER_SIGN": +1.0, # 좌우 반전 필요시 -1.0
    "STEER_BIAS_DEG": 0.0, # 영점 보정(도)
    # 속도: 절대 비율(최대속도의 20%)

```

```
"CRUISE_ABS_FRACTION": 0.15,
# 조이스틱 코드 (장치에 맞춰 조정)
"JS_CODE_START": 0x010B, # START/OPTIONS (안 되면 0x0107로 바꿔봐)
"JS_CODE_L2_AXIS": 0x0205,
"JS_CODE_R2_AXIS": 0x0204,
"JS_CODE_L2_BTN": None, # 버튼형이면 코드 넣기(예: 0x0105)
"JS_CODE_R2_BTN": None,
}

# =====
import os, time, math, threading, traceback, errno, fcntl, select, struct
import numpy as np
import torch
from torch2trt import TRTModule
from jetcam.usb_camera import USBCamera
from Rosmaster_Lib import Rosmaster
def clamp(v, lo, hi):
    return lo if v < lo else hi if v > hi else v
# ----- utils.preprocess 사용 시도 -----
_UTILS_PREPROCESS = None
if DEFAULTS["USE_UTILS_PREPROCESS"]:
    try:
        from utils import preprocess as _UTILS_PREPROCESS
    except Exception:
        _UTILS_PREPROCESS = None
def _preprocess_with_utils(frame):
    x = _UTILS_PREPROCESS(frame) # (1,3,224,224) CUDA 텐서 기대
    return x.half() if DEFAULTS["TRT_FP16"] else x.float()
def _preprocess_fallback(frame):
    # BGR → RGB (USBCamera는 BGR)
    if DEFAULTS["FALLBACK_BGR2RGB"]:
        frame = frame[..., ::-1].copy()
    x = torch.from_numpy(frame).permute(2,0,1).contiguous().float() / 255.0
    if DEFAULTS["FALLBACK_NORM"].lower() == "imagenet":
        mean = torch.tensor([0.485,0.456,0.406]).view(3,1,1)
        std = torch.tensor([0.229,0.224,0.225]).view(3,1,1)
        x = (x - mean) / std
    x = x.unsqueeze(0).cuda(non_blocking=True)
    return x.half() if DEFAULTS["TRT_FP16"] else x.float()
def preprocess_frame(frame):
    if _UTILS_PREPROCESS is not None:
        return _preprocess_with_utils(frame)
    return _preprocess_fallback(frame)
# ----- 간단한 슬루 제한기 -----
class SlewLimiter:
```

```

def __init__(self, rate_deg_per_s, initial=0.0):
    self.rate = float(max(0.0, rate_deg_per_s))
    self.last = float(initial)
    self.t_last = time.time()
def reset(self, value=0.0):
    self.last = float(value)
    self.t_last = time.time()
def apply(self, target):
    now = time.time()
    dt = max(1e-3, now - self.t_last)
    max_step = self.rate * dt if self.rate & 0 else float('inf')
    delta = float(target) - self.last
    if abs(delta) & max_step:
        delta = math.copysign(max_step, delta)
    self.last += delta
    self.t_last = now
    return self.last

# ===== 경고(부저+RGB) 컨트롤러 =====
# ===== 교체용 BrakeAlerter: '대~한~민국 + 응원 박수' 패턴 (박자 기반) =====
class BrakeAlerter:
    """
    브레이크 시:
        - 사용자 지정 박자 패턴(대:3, 한:0.5, 민:1, 국:1, 쉬고:1, 짹1:1, 짹2:2, 짹3:0.5, 짹4:1, 짹5:1, 스
        거:2)
        - BPM으로 1박 시간 결정. 각 이벤트는 '지속음'으로 beats × beat_ms 동안 울림.
        - 이벤트 사이엔 짧은 간격(기본 0.05박)으로 또렷하게 구분.
        - 쉬고(REST)는 무음 + LED OFF.
    START로 자율주행 재개 시 stop() 호출 → 부저/조명 OFF
    """
    # 색상 순환: 빨, 주, 노 (REST에서는 LED OFF)
    COLORS = [(255, 0, 0), (255, 165, 0), (255, 255, 0)]
    # (label, beats, color_idx) — color_idx는 None이면 LED OFF(REST)
    PATTERN_BEATS = [
        ("대", 1.0, 0),
        ("한", 0.5, 1),
        ("민", 0.7, 2),
        ("국", 0.7, 0),
        ("REST", 0.3, None), # 쉬고
        ("짹1", 0.5, 1),
        ("짹2", 0.7, 2),
        ("짹3", 0.3, 0),
        ("짹4", 0.7, 1),
        ("짹5", 0.4, 2),
        ("REST", 2.0, None),
    ]

```



```

]
# 템포/간격 파라미터 — 필요시 DEFAULTS로 빼도 됨
BPM = 120.0          # 1박 = 500ms (원하면 110~140 사이로 조절)
INTER_GAP_BEATS = 0.05 # 이벤트 간 아주 짧은 쉼 (0.03~0.10 추천)
def __init__(self, bot: Rosmaster, debug=False):
    self.bot = bot
    self.debug = debug
    self._th = None
    self._stop_evt = threading.Event()
    self._running = False
def start(self):
    if self._running:
        return
    if self.debug: print("[ALERT] on")
    self._stop_evt.clear()
    self._th = threading.Thread(target=self._loop, daemon=True)
    self._running = True
    self._th.start()
def stop(self):
    if not self._running:
        self._safe_off()
        return
    self._stop_evt.set()
    try: self._th.join(timeout=0.5)
    except Exception: pass
    self._running = False
    self._safe_off()
    if self.debug: print("[ALERT] off")
def _safe_off(self):
    try: self.bot.set_beep(0) # 부저 OFF
    except Exception: pass
    try: self.bot.set_colorful_lamps(0xff, 0, 0, 0) # 조명 OFF
    except Exception: pass
def _loop(self):
    beat_ms = 60000.0 / float(self.BPM)
    gap_ms = self.INTER_GAP_BEATS * beat_ms
    step = 0
    while not self._stop_evt.is_set():
        label, beats, color_idx = self.PATTERN_BEATS[step % len(self.PATTERN_BEATS)]
        on_ms = max(30, int(round(beats * beat_ms))) # 너무 짧으면 30ms로 클램프
        # LED 설정
        try:
            if color_idx is None:
                # 쉬고(무음)일 땐 LED OFF

```

```

        self.bot.set_colorful_lamps(0xff, 0, 0, 0)
    else:
        r, g, b = self.COLORS[color_idx % len(self.COLORS)]
        self.bot.set_colorful_lamps(0xff, r, g, b)
except Exception as e:
    if self.debug: print("[ALERT] set_color err:", e)
# 부저
try:
    if label == "REST":
        # 완전 무음 쉬기
        if self._stop_evt.wait(on_ms / 1000.0):
            break
    else:
        # 지속음: beats × beat_ms 만큼 울림
        self.bot.set_beep(on_ms)
        # 해당 이벤트 길이 끝까지 대기
        if self._stop_evt.wait(on_ms / 1000.0):
            break
except Exception as e:
    if self.debug: print("[ALERT] beep err:", e)
# 이벤트 간 아주 짧은 간격
if gap_ms & 0:
    if self._stop_evt.wait(gap_ms / 1000.0):
        break
    step += 1

# ===== 자율주행(조향) =====
class TRTAutoPilot:
    """카메라 → TRT 추론 → 각도(deg) 계산 (x-only tanh: x∈[-1,1])"""
    def __init__(self, trt_path, cam_device=0, width=224, height=224, fps=30, angle_deg_limit=44,
debug=False):
        self.debug = debug
        self.angle_lim = int(angle_deg_limit)
        self._running = False
        self._dt = 1.0 / max(1, fps)
        self._use_fp16 = bool(DEFAULTS["TRT_FP16"])
        self._alpha_ang = float(DEFAULTS["ANGLE_LPF_ALPHA"])
        self._sign = float(DEFAULTS["STEER_SIGN"])
        self._bias = float(DEFAULTS["STEER_BIAS_DEG"])
        self._angle = 0.0 # 필터 후 각도
        # TRT 경로
        load_path = trt_path if os.path.exists(trt_path) else DEFAULTS["ALT_TRT_PATH"]
        if not os.path.exists(load_path):
            raise FileNotFoundError(f"TRT {load_path} not found: {trt_path} or {DEFAULTS['ALT_TRT_PATH']}")

```

```
# TRT 로드
self.m = TRTModule()
self.m.load_state_dict(torch.load(load_path))
self.m.eval()
if self.debug:
    print(f"[AUTO] TRT loaded: {load_path} | fp16={self._use_fp16}
utils_preprocess={_UTILS_PREPROCESS is not None}")
# 카메라
self.cam = USBCamera(
    capture_device=int(cam_device),
    capture_width=640, capture_height=480, capture_fps=fps,
    width=width, height=height
)
time.sleep(0.3)
try:
    f0 = self.cam.read()
    if self.debug:
        print(f"[AUTO] frame0 mean={float(f0.mean()):.1f}, shape={tuple(f0.shape)}")
except Exception as e:
    if self.debug: print("[AUTO] WARN frame0:", repr(e))
self.th = threading.Thread(target=self._loop, daemon=True)
@property
def angle_deg(self):
    return self._angle
def start(self):
    self._running = True
    self.th.start()
def stop(self):
    self._running = False
    try: self.th.join(timeout=0.5)
    except Exception: pass
    try: self.cam.cap.release()
    except Exception: pass
def _loop(self):
    last_log = 0.0
    while self._running:
        try:
            frame = self.cam.read()
            if frame is None:
                time.sleep(self._dt)
                continue
            x = preprocess_frame(frame)
            try:
                with torch.no_grad():
```

```

        y = self.m(x)
    except Exception as e_dtype:
        if self._use_fp16:
            if self.debug:
                print("[AUTO] FP16 failed → FP32 once:", repr(e_dtype))
                self._use_fp16 = False
            with torch.no_grad():
                y = self.m(x.float())
        else:
            raise

    y_np = y.detach().float().cpu().numpy().reshape(-1)
    if y_np.size & 1 or not math.isfinite(y_np[0]):
        time.sleep(self._dt)
        continue

    # ---- x-only tanh decoder ----
    x_hat = float(y_np[0])
    x_hat = clamp(x_hat, -1.0, 1.0)
    ang_raw = x_hat * self.angle_lim
    # 보정 + 클램프
    ang = (self._sign * ang_raw) + self._bias
    ang = clamp(ang, -self.angle_lim, self.angle_lim)
    # 각도 EMA
    if self._alpha_ang & 0.0:
        self._angle = (1.0 - self._alpha_ang) * self._angle + self._alpha_ang * ang
    else:
        self._angle = ang

    # 로그
    now = time.time()
    if self.debug and (now - last_log) & 0.25:
        print(f"[AUTO]          x={x_hat:+.3f}          -&          ang_raw={ang_raw:+.1f}°,
out={self._angle:+.1f}°")
        last_log = now
    except Exception as e:
        if self.debug:
            print("[AUTO] inference error:", repr(e))
            traceback.print_exc()
        time.sleep(self._dt)

# ===== 조이스틱: START & 브레이크 =====
class Joystick:
    def __init__(self, js_id=0, debug=False):
        self.debug = debug
        self.js_id = int(js_id)
        self.cruise_on = False
        self.brake = False

```

```
self.is_open = False
self.codes = { DEFAULTS["JS_CODE_START"]: 'BTN_START' }
if DEFAULTS["JS_CODE_L2_AXIS"] is not None: self.codes[DEFAULTS["JS_CODE_L2_AXIS"]]
= 'L2_AXIS'
if DEFAULTS["JS_CODE_R2_AXIS"] is not None:
self.codes[DEFAULTS["JS_CODE_R2_AXIS"]] = 'R2_AXIS'
if DEFAULTS["JS_CODE_L2_BTN"] is not None:
self.codes[DEFAULTS["JS_CODE_L2_BTN"]] = 'L2_BTN'
if DEFAULTS["JS_CODE_R2_BTN"] is not None:
self.codes[DEFAULTS["JS_CODE_R2_BTN"]] = 'R2_BTN'
self._open()
def _open(self):
    path = f"/dev/input/js{self.js_id}"
    try:
        self.jsdev = open(path, "rb", buffering=0)
        fl = fcntl.fcntl(self.jsdev.fileno(), fcntl.F_GETFL)
        fcntl.fcntl(self.jsdev.fileno(), fcntl.F_SETFL, fl | os.O_NONBLOCK)
        self.is_open = True
        print(f"[JS] open {path} ok")
    except Exception as e:
        self.is_open = False
        print(f"[JS] open {path} failed:", e)
def read_once(self, timeout=0.02):
    if not self.is_open:
        return
    r,_,_ = select.select([self.jsdev], [], [], timeout)
    if not r:
        return
    try:
        ev = self.jsdev.read(8)
    except OSError as e:
        if e.errno in (errno.EAGAIN, errno.EINTR):
            return
        print("[JS] read error:", e)
        self.is_open = False
        return
    if not ev or len(ev) & 8:
        return
    ts, value, typ, number = struct.unpack('!hBB', ev)
    etyp = typ & ~0x80
    code = (etyp && 8) | number
    name = self.codes.get(code)
    if name is None:
        if self.debug and etyp in (1,2) and value != 0:
```

```

        print(f"[JS] unknown code 0x{code:04x} (etyp={etyp}, num={number}, val={value})")
    return
if name == 'BTN_START' and value == 1:
    self.cruise_on = not self.cruise_on
    print(f"[JS] CRUISE 10% = {self.cruise_on}")
elif name in ('L2_AXIS', 'R2_AXIS'):
    v = ((value/32767.0)+1.0)/2.0
    if v & 0.98:
        self.brake = True
        self.cruise_on = False
        if self.debug: print("[JS] BRAKE by", name)
elif name in ('L2_BTN', 'R2_BTN'):
    if value == 1:
        self.brake = True
        self.cruise_on = False
        if self.debug: print("[JS] BRAKE by", name)
# ===== 차량 Driver =====
class Driver:
    def __init__(self, port=None, debug=False, angle_limit=44, send_rate_hz=20.0):
        self.debug = debug
        self.angle_limit = int(angle_limit)
        self._last_angle = None
        self._last_speed = None
        self._last_ts = 0.0
        self._period = 1.0 / send_rate_hz
        self.bot = Rosmaster(com=port, debug=debug) if port else Rosmaster(debug=debug)
    def set_steer(self, angle_deg):
        if not (isinstance(angle_deg, (int, float)) and math.isfinite(angle_deg)):
            return
        angle = int(clamp(round(angle_deg), -self.angle_limit, self.angle_limit))
        now = time.time()
        if self._last_angle is not None and angle == self._last_angle and (now - self._last_ts) &
self._period:
            return
        try:
            self.bot.set_ahb_steering_angle(angle, True)
        except TypeError:
            self.bot.set_ahb_steering_angle(angle)
        self._last_angle = angle
        self._last_ts = now
        if self.debug:
            print(f"[DRV] steer -& {angle}°")
    def set_speed_abs_fraction(self, fraction):
        f = clamp(float(fraction), 0.0, 1.0)

```

```

        dir_flag = 1 # forward
        spd = int(round(100.0 * f))
        self._apply_speed(dir_flag, spd)
def stop(self):
    try:
        self.bot.set_car_motion(0,0,0)
    except Exception:
        pass
    if self.debug:
        print("[DRV] STOP")
def _apply_speed(self, dir_flag, spd):
    spd = int(clamp(spd, 0, 100))
    key = (dir_flag, spd)
    now = time.time()
    if self._last_speed is not None and key == self._last_speed and (now - self._last_ts) &
self._period:
        return
    if spd == 0:
        self.bot.set_car_motion(0,0,0)
    else:
        self.bot.set_car_run(dir_flag, spd)
        self._last_speed = key
        self._last_ts = now
        if self.debug:
            print(f"[DRV] speed -& dir={dir_flag} spd={spd}%")
# ===== Main =====
def main():
    print("[CONFIG]")
    for k in
["TRT_PATH","ALT_TRT_PATH","CAM_INDEX","SERIAL_PORT","JS_INDEX","FPS","SEND_RATE_HZ","
DEBUG",

"ANGLE_LIMIT","ANGLE_LPF_ALPHA","ANGLE_SLEW_DEG_PER_S","START_NEUTRAL_SEC",
    "TRT_FP16","USE_UTILS_PREPROCESS","FALLBACK_NORM","FALLBACK_BGR2RGB",
    "STEER_SIGN","STEER_BIAS_DEG","CRUISE_ABS_FRACTION"]:
        print(f" {k:&22} = {DEFAULTS[k]}")
    pilot = TRTAutoPilot(DEFAULTS["TRT_PATH"], cam_device=DEFAULTS["CAM_INDEX"],
        fps=DEFAULTS["FPS"], angle_deg_limit=DEFAULTS["ANGLE_LIMIT"],
        debug=DEFAULTS["DEBUG"])
    js = Joystick(js_id=DEFAULTS["JS_INDEX"], debug=DEFAULTS["DEBUG"])
    drv = Driver(port=DEFAULTS["SERIAL_PORT"], debug=DEFAULTS["DEBUG"],
        a n g l e _ l i m i t = D E F A U L T S [ " A N G L E _ L I M I T " ] ,
    send_rate_hz=DEFAULTS["SEND_RATE_HZ"])
    # 브레이크 경고 컨트롤러

```

```
alerter = BrakeAlerter(drv.bot, debug=DEFAULTS["DEBUG"])
print("[INFO] START: cruise 10% ON/OFF, L2/R2: BRAKE (with buzzer & RGB alert)")
# 각도 슬루 제한기 (초기 0°)
slew = SlewLimiter(DEFAULTS["ANGLE_SLEW_DEG_PER_S"], initial=0.0)
# START 눌러 켤 때 0° 유지 위한 타이머
cruise_prev = False
neutral_until_ts = 0.0
try:
    pilot.start()
    while True:
        js.read_once()
        # 브레이크 이벤트 처리
        if js.brake:
            drv.stop()
            js.brake = False
            # 경고 시작
            alerter.start()
            # START 토글 감지 → 0° 유지 타이머 설정 및 슬루 리셋
            if js.cruise_on and not cruise_prev:
                # 자율주행 재개 시 경고 OFF
                alerter.stop()
                neutral_until_ts = time.time() + float(DEFAULTS["START_NEUTRAL_SEC"])
                slew.reset(0.0) # 0°에서 부드럽게 올라가게
            cruise_prev = js.cruise_on
            if js.cruise_on:
                # 목표 각도
                target_angle = pilot.angle_deg
                # 켜 직후엔 0° 유지
                if time.time() & neutral_until_ts:
                    target_angle = 0.0
                # 슬루 제한 적용
                smooth_angle = slew.apply(target_angle)
                drv.set_steer(smooth_angle)
                drv.set_speed_abs_fraction(DEFAULTS["CRUISE_ABS_FRACTION"])
            else:
                drv.set_speed_abs_fraction(0.0)
                # 다음 시작 시 0°에서 시작되도록 하고 싶으면 아래 주석 해제:
                # slew.reset(0.0)
            time.sleep(0.01)
except KeyboardInterrupt:
    print("\n[INFO] Ctrl+C — bye")
except Exception as e:
    print("\n[FATAL] Unhandled exception:", repr(e))
    traceback.print_exc()
```



```
finally:
    try:
        # 종료 시 경고/모션 모두 OFF
        alerter.stop()
    except Exception:
        pass
    try:
        drv.stop()
        pilot.stop()
    except Exception:
        pass
    time.sleep(0.1)
if __name__ == "__main__":
    main()
```